

ROTEIRO DE LABORATÓRIO 2

Refatorando um relatório de vendas: do procedural ao funcional

Aula 2, 18 de agosto · duração prevista: 2h30

O que você vai construir

Este roteiro parte de um script que funciona e é difícil de mudar, e o transforma em um conjunto de funções puras compostas em sequência, sem alterar o resultado.

A refatoração é feita com rede de proteção: os testes vêm antes de qualquer alteração. Essa ordem não é detalhe de estilo. Refatorar sem teste é apostar que nada quebrou, e a aposta costuma sair cara em código que já está em produção.

AO FINAL DESTES ROTEIRO VOCÊ TERÁ

Um script procedural funcionando, com a saída conhecida e registrada.

Uma suíte de testes com pelo menos quatro casos, passando.

O mesmo relatório reescrito em funções puras compostas em pipeline.

Um documento comparando as duas versões, com a decisão da equipe justificada.

Antes de começar

Tudo isto foi resolvido no roteiro anterior. Se algum item falhar, resolva primeiro, porque o restante depende dele.

- Python 3.12 ou superior instalado e verificado.
- O repositório da equipe clonado na sua máquina.
- VS Code com a extensão Python.

Passo a passo

Trabalhem em equipe, mas cada integrante digita na própria máquina. Uma pessoa apenas assistindo não aprende a refatorar.

1

Preparar a pasta e o ambiente isolado 10 min

Dentro do repositório da equipe, crie a pasta deste laboratório. Em seguida crie um ambiente virtual, que mantém as bibliotecas do projeto separadas das do sistema.

NO TERMINAL

```
cd ads-2026-2-biblioteca
mkdir aula02-paradigmas
cd aula02-paradigmas
```

```
# criar o ambiente virtual
python -m venv .venv
```

```
# ativar, no Windows:
.venv\Scripts\activate
```

```
# ativar, no macOS ou Linux:
source .venv/bin/activate

# instalar a ferramenta de testes
pip install pytest
```

SAÍDA ESPERADA

```
(.venv) C:\...\aula02-paradigmas>

# o (.venv) no início da linha indica que o ambiente está ativo
Successfully installed pytest-8.3.2
```

POR QUE ISSO IMPORTA

O ambiente virtual evita um problema clássico: dois projetos na mesma máquina precisando de versões diferentes da mesma biblioteca. Cada projeto passa a ter as suas. A pasta `.venv` não vai para o Git. Ela será ignorada no passo 6.

CONFIRA ANTES DE SEGUIR

O prompt do terminal mostra `(.venv)` no início da linha e o `pytest` foi instalado.

2

Criar o script procedural 15 min

Crie o arquivo `relatorio.py` com o conteúdo abaixo. Digite em vez de copiar: digitar obriga a ler cada linha, e ler é o objetivo deste passo.

```
RELATORIO.PY
VENDAS = [
    {"produto": "Teclado", "valor": 150.00, "categoria": "Periferico"},
    {"produto": "Mouse", "valor": 80.00, "categoria": "Periferico"},
    {"produto": "Monitor", "valor": 900.00, "categoria": "Tela"},
    {"produto": "Cabo HDMI", "valor": 35.00, "categoria": "Acessorio"},
    {"produto": "Headset", "valor": 250.00, "categoria": "Periferico"},
    {"produto": "Suporte", "valor": 120.00, "categoria": "Acessorio"},
]

IMPOSTO = 0.10
VALOR_MINIMO = 100.00

def relatorio(vendas):
    """Total líquido por categoria, apenas de vendas acima do mínimo."""
    total_por_categoria = {}

    for v in vendas:
        if v["valor"] <= VALOR_MINIMO:
            continue

        liquido = v["valor"] * (1 - IMPOSTO)
        cat = v["categoria"]

        if cat not in total_por_categoria:
            total_por_categoria[cat] = 0
        total_por_categoria[cat] += liquido

    return total_por_categoria
```

```
if __name__ == "__main__":
    for categoria, total in relatorio(VENDAS).items():
        print(f"{categoria:12} R$ {total:8.2f}")
```

NO TERMINAL

```
python relatorio.py
```

SAÍDA ESPERADA

Periferico	R\$	360.00
Tela	R\$	810.00
Acessorio	R\$	108.00

ATENÇÃO

Se a sua saída for diferente, confira os valores da lista VENDAS e o sinal do teste na linha do if. Mouse e Cabo HDMI ficam de fora porque valem 100 ou menos.

Registre essa saída. Ela é a referência contra a qual toda alteração será comparada.

CONFIRA ANTES DE SEGUIR

O script roda e imprime exatamente os três valores acima.

3

Escrever os testes antes de mexer 25 min

Este é o passo que torna a refatoração segura. Os testes descrevem o comportamento atual, e passam a acusar qualquer alteração que mude o resultado. Crie o arquivo `test_relatorio.py` na mesma pasta.

TEST_RELATORIO.PY

```
from relatorio import relatorio

def test_caso_comum():
    vendas = [
        {"produto": "A", "valor": 200.00, "categoria": "X"},
        {"produto": "B", "valor": 300.00, "categoria": "X"},
    ]
    assert relatorio(vendas) == {"X": 450.00}

def test_lista_vazia():
    assert relatorio([]) == {}

def test_nenhuma_venda_passa_do_minimo():
    vendas = [
        {"produto": "A", "valor": 50.00, "categoria": "X"},
        {"produto": "B", "valor": 100.00, "categoria": "X"},
    ]
    assert relatorio(vendas) == {}

def test_agrupa_por_categoria():
    vendas = [
```

```
        {"produto": "A", "valor": 200.00, "categoria": "X"},
        {"produto": "B", "valor": 200.00, "categoria": "Y"},
    ]
    assert relatorio(vendas) == {"X": 180.00, "Y": 180.00}
```

NO TERMINAL

```
pytest -v
```

SAÍDA ESPERADA

```
test_relatorio.py::test_caso_comum PASSED
test_relatorio.py::test_lista_vazia PASSED
test_relatorio.py::test_nenhuma_venda_passa_do_minimo PASSED
test_relatorio.py::test_agrupa_por_categoria PASSED
```

```
===== 4 passed in 0.03s =====
```

POR QUE ISSO IMPORTA

Repare no terceiro teste: ele fixa uma decisao de negocio. Uma venda de exatamente 100 reais fica de fora, porque a condicao usa menor ou igual.

Sem esse teste, alguem poderia trocar o operador durante a refatoracao e ninguem perceberia. Teste tambem serve para registrar decisao.

ATENÇÃO

Nao siga para o proximo passo com teste vermelho. Se algum falhar, o problema esta no relatorio.py ou na conta do teste, e precisa ser resolvido antes de refatorar.

CONFIRA ANTES DE SEGUIR

O pytest reporta 4 passed. Faça um commit agora, com a mensagem "adiciona testes do relatorio antes da refatoracao".

4

Separar as três responsabilidades 35 min

O laço atual faz três coisas ao mesmo tempo: decide o que entra, calcula o valor líquido e agrupa por categoria. Vamos extrair cada uma para a sua própria função. Rode os testes depois de cada extração, e não só ao final.

Extração 1: a regra de seleção

ACRESCENTE AO RELATORIO.PY

```
def acima_do_minimo(venda):
    """Diz se a venda entra no relatorio."""
    return venda["valor"] > VALOR_MINIMO
```

Agora use essa função dentro do laço, no lugar da comparação direta, e rode o pytest.

DENTRO DO LAÇO, SUBSTITUA

```
# antes:
if v["valor"] <= VALOR_MINIMO:
    continue

# depois:
if not acima_do_minimo(v):
```

```
continue
```

Extração 2: o cálculo do líquido

ACRESCENTE AO RELATORIO.PY

```
def aplicar_imposto(venda):
    """Devolve uma NOVA venda com o valor líquido.

    Não altera a venda recebida. Essa é a diferença
    entre uma função pura e uma que modifica no lugar.
    """
    return {**venda, "valor": venda["valor"] * (1 - IMPOSTO)}
```

POR QUE ISSO IMPORTA

A sintaxe `{**venda, "valor": ...}` cria um dicionário novo copiando tudo do original e trocando apenas a chave `valor`.

Poderia ser mais simples alterar `venda["valor"]` direto. O motivo de não fazer isso é que a lista original passaria a estar modificada, e qualquer outro cálculo sobre ela usaria valores já com imposto aplicado. Esse tipo de defeito é difícil de encontrar.

Extração 3: o agrupamento

ACRESCENTE AO RELATORIO.PY

```
def agrupar_por_categoria(accumulado, venda):
    """Soma a venda no total da sua categoria."""
    cat = venda["categoria"]
    return {**accumulado, cat: acumulado.get(cat, 0) + venda["valor"]}
```

NO TERMINAL

```
pytest -v
```

SAÍDA ESPERADA

```
===== 4 passed in 0.03s =====
```

CONFIRA ANTES DE SEGUIR

As três funções existem, o laço original ainda funciona e os 4 testes continuam passando.

5 Compor as funções em um pipeline 30 min

Com as três responsabilidades isoladas, a função principal deixa de ter um laço e passa a declarar a sequência de transformações. Substitua o corpo de relatório pelo código abaixo.

NOVA VERSÃO DE RELATORIO

```
from functools import reduce
```

```
def relatorio(vendas):
    """Total líquido por categoria, apenas de vendas acima do mínimo."""
    relevantes = filter(acima_do_minimo, vendas)
    liquidas = map(aplicar_imposto, relevantes)
    return reduce(agrupar_por_categoria, liquidas, {})
```

A linha do import precisa ficar no topo do arquivo, junto das outras importações. Rode os testes e o script.

```
NO TERMINAL
pytest -v
python relatorio.py
```

SAÍDA ESPERADA

```
===== 4 passed in 0.03s =====

Periferico    R$    360.00
Tela          R$    810.00
Acessorio     R$    108.00
```

ATENÇÃO

A saída precisa ser idêntica a do passo 2. Se algum número mudou, alguma extração alterou o comportamento e os testes deveriam ter acusado. Volte e confira.

POR QUE ISSO IMPORTA

Tres verbos resolvem a maior parte das transformações de dados na profissão: filter seleciona, map transforma, reduce acumula.

O vocabulário se repete em outras tecnologias. JavaScript usa filter, map e reduce. Java usa stream com filter, map e collect. SQL faz o mesmo com WHERE, SELECT e GROUP BY.

CONFIRA ANTES DE SEGUIR

A função relatorio não tem mais nenhum laço for, os 4 testes passam e a saída é idêntica à do passo 2.

6

Provar que a versão nova é mais fácil de mudar 20 min

Refatoração só se justifica se o código ficou melhor para alguma coisa. Vamos testar isso com uma mudança de requisito real: o imposto passa a variar por categoria.

A nova regra

- Periferico: 10 por cento.
- Tela: 15 por cento.
- Qualquer outra categoria: 5 por cento.

Implemente essa mudança alterando apenas a função aplicar_imposto. Escreva antes um teste que descreva a nova regra, veja o teste falhar, e só então altere a função.

ACRESCENTE AO TEST_RELATORIO.PY

```
def test_imposto_varia_por_categoria():
    vendas = [
        {"produto": "A", "valor": 200.00, "categoria": "Tela"},
    ]
    # 15% de imposto sobre 200 deixa 170
    assert relatorio(vendas) == {"Tela": 170.00}
```

```
NO TERMINAL
pytest -v
```

SAÍDA ESPERADA ANTES DA CORREÇÃO

```
test_relatorio.py::test_imposto_varia_por_categoria FAILED

E   assert {"Tela": 180.0} == {"Tela": 170.0}

===== 1 failed, 4 passed in 0.04s =====
```

Agora altere a função. A equipe deve conseguir fazer isso mexendo em um único lugar.

POR QUE ISSO IMPORTA

Escrever o teste antes e vê-lo falhar tem um propósito: prova que o teste realmente testa alguma coisa. Um teste que passa desde o início pode não estar verificando nada. Anote quanto tempo a equipe levou nesta mudança. Esse número entra no documento final.

CONFIRA ANTES DE SEGUIR

Os 5 testes passam e a alteração foi feita em uma única função.

7

Escrever a análise e enviar 15 min

Crie o arquivo REFATORACAO.md na pasta do laboratório, respondendo às perguntas abaixo. Guarde as duas versões da função relatorio no documento, para comparação.

ESTRUTURA DO REFATORACAO.MD

```
# Refatoracao do relatorio de vendas

## As duas versoes
(cole aqui a versao com laco e a versao com pipeline)

## Tempo gasto
- Escrever os testes: __ minutos
- Separar as tres funcoes: __ minutos
- Aplicar a mudanca de imposto por categoria: __ minutos

## Qual versao a equipe leva para o projeto do semestre
(cinco linhas, citando pelo menos um criterio concreto:
  facilidade de teste, facilidade de mudanca, ou legibilidade)

## O que ficou em duvida
(uma pergunta que a equipe quer ver respondida)
```

Ignorar o que não deve ir para o Git

CRIE O ARQUIVO .GITIGNORE NA RAIZ DO REPOSITÓRIO

```
.env/
__pycache__/
.pytest_cache/
*.pyc
```

NO TERMINAL

```
git add .
git status # confira que .env NAO aparece na lista
git commit -m "refatora relatorio para pipeline funcional com testes"
git push
```

ATENÇÃO

Se a pasta `.venv` aparecer no git status, o `.gitignore` não está sendo lido. Confira se ele está na raiz do repositório e se o nome começa com ponto.

Enviar a `.venv` por engano deixa o repositório com milhares de arquivos inúteis.

CONFIRA ANTES DE SEGUIR

O push foi aceito e, ao abrir o repositório no navegador, aparecem `relatorio.py`, `test_relatorio.py` e `REFATORACAO.md`, sem a pasta `.venv`.

Se der errado

Os problemas abaixo respondem pela maior parte das travadas neste roteiro.

Sintoma	O que fazer
<code>ModuleNotFoundError: relatorio</code>	O pytest foi executado de outra pasta. Entre na pasta aula02-paradigmas antes de rodar.
<code>pytest: command not found</code>	O ambiente virtual não está ativo. Rode o comando de ativação do passo 1 e confirme que aparece (.venv) no prompt.
<code>assert com valor tipo 494.99999</code>	É o arredondamento de ponto flutuante. Para este laboratório, use <code>round(valor, 2)</code> na comparação. O assunto é tratado a fundo na próxima aula.
<code>TypeError: reduce not defined</code>	Faltou a linha <code>from functools import reduce</code> no topo do arquivo.
<code>O map e o filter devolvem objeto estranho</code>	Em Python 3 eles devolvem um iterador preguiçoso, e não uma lista. Dentro do <code>reduce</code> isso funciona normalmente. Para inspecionar, envolva com <code>list()</code> .
<code>Os testes passam mas a saída mudou</code>	Os testes não cobrem o caso que quebrou. Acrescente um teste que reproduza a diferença antes de corrigir.

Desafios opcionais

Para quem terminou antes do fim do laboratório. Não valem nota, mas costumam aparecer na prova em alguma forma.

- Reescreva o pipeline usando compreensão de lista no lugar de `map` e `filter`. Compare a legibilidade das duas formas e diga qual a equipe prefere.
- Faça a mesma refatoração em JavaScript, usando os métodos `filter`, `map` e `reduce` de `Array`. Compare o resultado com a versão em Python.
- Acrescente uma função que devolva a categoria de maior faturamento, sem alterar nenhuma das funções existentes. Se for preciso alterar alguma, discuta por que o desenho anterior não estava suficientemente separado.
- Pesquise a diferença entre uma função pura e uma função com efeito colateral, e classifique cada uma das quatro funções do seu arquivo.

Entrega

O que entregar	A pasta aula02-paradigmas no repositório, com <code>relatorio.py</code> , <code>test_relatorio.py</code> e <code>REFATORACAO.md</code>
Onde entregar	No próprio repositório da equipe, com commit feito ainda em aula
Quem entrega	A equipe, com commits de todos os integrantes
Prazo	Até o final do encontro

Critério de aceite

- Os testes rodam e passam com o comando `pytest`.
- A função `relatorio` final não contém `for`.
- A saída do script é idêntica à registrada no passo 2.
- O `REFATORACAO.md` responde qual versão a equipe adota, com pelo menos um critério concreto.
- A pasta `.venv` não foi enviada ao repositório.

Para consultar durante o laboratório

Martin Fowler, Collection Pipeline

<https://martinfowler.com/articles/collection-pipeline/>

Python, tutorial oficial sobre programação funcional

<https://docs.python.org/pt-br/3/howto/functional.html>

Documentação do pytest, primeiros passos

<https://docs.pytest.org/en/stable/getting-started.html>

Python, ambientes virtuais com venv

<https://docs.python.org/pt-br/3/library/venv.html>

Paul Graham, Beating the Averages

<https://paulgraham.com/avg.html>